University of Engineering and Technology, New Campus, Lahore
Department of Computer Science

Course Code & Title: CSC-200L: Data Structures and Algorithms Lab

Total Marks: 40 **Open Ended Lab** **Deadline:** 7-June-2026

1. Schwab/Herron Levels of Laboratory Openness

Level	Methodology	Description	Problem	Ways & Means	Answers
2	Guided Inquiry	Students investigate an instructor-defined computational problem using self-designed data structures and algorithms.	Given	Open	Open

2. Problem Statement

Design and Implementation of a Real-Time Food Delivery Optimization and Dispatch Engine

Problem Description

FoodExpress is a large-scale food delivery platform operating across a busy metropolitan city. During peak hours, the platform receives thousands of food orders from different customers and restaurants simultaneously. The company is facing several computational and operational challenges including dynamic and continuous order arrivals, priority-based order processing, overloaded restaurant preparation queues, inefficient rider assignment, changing delivery routes due to traffic conditions, large-scale order tracking, and performance degradation as the dataset grows.

Your task is to design and develop a Data Structure and Algorithm driven **Dispatch Engine** that focuses on solving these optimization and processing problems efficiently. The objective of this project is not to build a simple CRUD-based application; instead, the system should demonstrate meaningful use of efficient data organization, intelligent scheduling, searching and retrieval optimization, route optimization, dynamic processing, load balancing, and performance analysis. Students are free to choose their own data structures and algorithms according to their design approach. However, every design choice must be properly justified in the report and presentation.

The system should include a **Dynamic Order Scheduling Engine** capable of managing dynamically arriving food orders with different priorities, preparation times, delivery deadlines, customer categories, and restaurant workloads. The module should support insertion of new orders, cancellation of orders, updating order priorities, delayed order handling, dynamic rescheduling, and retrieval of the next processable order.

The project should also contain a **Kitchen Load Balancing Module** where restaurants have limited preparation capacity. The system should efficiently distribute incoming orders, detect overloaded kitchens, rebalance workload dynamically, estimate waiting times, and optimize order processing flow.

A Rider Dispatch and Assignment Optimization Module should be developed to manage multiple riders operating simultaneously. Rider assignment should consider factors such as current location, delivery load, availability, delivery capacity, and route distance. The system should identify suitable riders, minimize delivery delays, balance rider workload, and optimize dispatch decisions.

The system should further implement a **Real-Time Route Optimization Module** in which the city map is represented computationally as a connected network of locations. The module should support shortest route calculation, route comparison, rerouting when roads are blocked, and delivery cost estimation. Students are free to choose suitable graph representations and routing strategies.

An efficient **Search and Retrieval Engine** should also be implemented for managing and searching orders, customers, restaurants, riders, and delivery records. The system should support fast lookup operations, category-based searching, filtered searching, and efficient record updates.

The project should additionally include an **Order History and Tracking Module** capable of maintaining complete order history and tracking state transitions such as placed, accepted, queued, prepared, assigned, picked, delivered, delayed, cancelled, or rerouted. The module should also support replaying order timelines and undo operations.

Finally, students must develop a **Performance Analysis Module** to evaluate insertion performance, deletion performance, searching efficiency, scheduling efficiency, route calculation performance, memory usage, and scalability behavior. The analysis should include theoretical complexity discussion, experimental evaluation, comparisons using different dataset sizes, and justification of optimization decisions.

3. Instructions

- Each team must consist of 4 students.
- The project must be implemented CLI/menu based in C++.
- Students must implement custom logic for their selected approaches.
- Every selected data structure and algorithm must be properly justified.
- Complexity analysis is compulsory.
- Ethical and professional handling of customer data must be discussed.
- **Built-in implementations of major data structures are not allowed.**

4. Deliverables

Each team must submit the following:

A. UML / Design Documentation

Include:

- Module based full working Code

- Class Diagrams
- Flowcharts
- Architecture Diagrams
- Module Design

B. Additional Submission Material

- Screenshots or video demonstration , LinkedIn post link

5. Suggested System Menu

FoodExpress Dispatch Optimization Engine

1. Dynamic Order Scheduling
2. Kitchen Load Analysis
3. Rider Dispatch Optimization
4. Route Optimization
5. Search and Retrieval Engine
6. Order History Tracking
7. Performance Analysis
8. Scalability Simulation
9. System Reports
10. Exit

6. Open Ended Lab Assessment Rubrics

Criteria	Excellent (5)	Good (4)	Satisfactory (3)	Needs Improvement (2)	Poor (1)
Data Structure Selection & Implementation (CLO1, CLO3)	Efficient and well-justified use of multiple data structures with strong custom implementation	Appropriate structures used with minor issues	Basic structures implemented correctly	Limited or weak implementation	Incorrect or minimal implementation
Algorithm Design & Optimization	Highly optimized algorithms with strong scheduling,	Good optimization with	Functional algorithms with limited optimization	Weak or inefficient algorithms	Poor or incorrect algorithm design

(CLO2, CLO3)	routing, searching, and balancing strategies	reasonable efficiency			
Complexity & Performance Analysis (CLO2)	Complete complexity analysis with experimental comparison and scalability testing	Good analysis with minor gaps	Basic complexity discussion included	Weak or incomplete analysis	No meaningful analysis
System Functionality & Integration (CLO3)	All modules fully functional and well-integrated	Most modules functional	Basic modules working	Partial implementation	Mostly incomplete
Code Quality & Documentation	Clean, modular, readable, and properly documented code with diagrams	Good organization and documentation	Acceptable structure with limited documentation	Poor organization/documentation	No proper documentation
Innovation & Problem Solving	Creative computational solutions with effective optimization decisions	Some innovative approaches	Standard implementation	Minimal optimization effort	No meaningful problem solving
Professional Practices (CLO4)	Proper discussion of ethics, scalability, and responsible data handling	Basic professional considerations included	Limited discussion	Weak understanding	No discussion